Devoid Viewer

Codebase & Installer Documentation

This document explains every file, struct, method, and design decision in the Devoid Viewer codebase, and documents the Inno Setup installer. Written for someone new to Rust who wants to understand not only *what* the code does but *why* it was written that way.

Table of Contents

Section	Topic	Page
1	Project structure overview	2
2	Cargo.toml — dependencies explained	2
3	nav.rs — directory navigation	4
4	loader.rs — image decoding	7
5	main.rs — application core	11
6	Threading & asynchronous design	23
7	Key design decisions & trade-offs	25
8	Caveats & known limitations	28
9	Installer — devoid_viewer_setup.iss	28
10	Glossary of Rust concepts used	31

1. Project Structure Overview

The project follows the standard Cargo layout:

File	Responsibility
`Cargo.toml`	Project name, version, and all external dependencies.
`build.rs`	Embeds devoid.ico and version metadata into the exe via winres.
`src/main.rs`	Event loop, app state, input handling, pixel renderer, window management.
`src/loader.rs`	Image decoding for all supported formats. Owns Frame and DecodedImage.
`src/nav.rs`	Directory scanning, human-sorted sibling list, next/prev navigation.
`devoid_viewer_setup.iss`	Inno Setup 6 installer script.
`devoid.ico`	Multi-size application icon. Embedded in the exe at compile time.

Supported formats: BMP, PNG, JPEG, GIF (animated), WebP, TIFF, ICO, JPEG XL. SVG support was intentionally removed to keep the binary ultra-lightweight.

2. Cargo.toml — Dependencies Explained

Each crate was chosen for leanness. The dependency graph is roughly 50–70 crates and the release binary around 2–3 MB.

Windowing: winit

winit handles the event loop, keyboard, window resize, drag-and-drop, and monitor queries. The rwh_05 feature enables the raw-window-handle v0.5 interface that softbuffer requires. On Windows the platform extension EventLoopBuilderExtWindows is used to install a Win32 message hook.

Pixel display: softbuffer

softbuffer provides a Vec<u32> that maps directly to the window's pixels. No GPU pipeline, no shader compilation. On Windows it uses a GDI DIB section accelerated by the compositor.

> ■ *DWM fills every new window with white before the application paints. Mitigated by hiding the window until the image is ready and calling paint_background() synchronously before the event loop starts.*

Native dialogs: windows-sys

windows-sys is a zero-overhead Windows API binding. Used for MessageBoxW (all dialogs), SystemParametersInfoW(SPI_GETWORKAREA) (precise taskbar exclusion), and PostMessageW (taskbar thumbnail close fix).

Image decoding: image

The **image** crate handles BMP, PNG, JPEG, GIF, WebP, TIFF, and ICO. GIF animation uses GifDecoder with AnimationDecoder. The ICO format is also used to decode devoid.ico for the window icon.

JPEG XL: jxl-oxide

Pure-Rust JPEG XL decoder. Not yet part of the image crate. Uses the low-level streaming API for frame rendering.

Channel communication: crossbeam-channel

Faster and more ergonomic than std::sync::mpsc. Bounded at capacity 32 to throttle background threads if the UI falls behind.

Windows resource embedding: winres (build dependency)

Runs at build time only. Embeds devoid.ico and version metadata into the exe. Zero runtime cost.

3. nav.rs — Directory Navigation

Maintains a human-sorted list of sibling image files in the same directory as the currently open file. Navigation wraps at both ends.

struct DirNavigator — fields

- **files: Vec<PathBuf>** — human-sorted sibling list.
- **index: usize** — position of the currently displayed file.
- **current_dir: Option<PathBuf>** — the directory files was built from. Stored explicitly to avoid inferring it from files.first(), which gives wrong answers after the single-entry fallback resets the list.

fn set_current(path)

Compares the new path's parent against current_dir. Rebuilds only on change. After rebuilding, locates path in the sorted list with position(). The fallback (replacing the list with a single entry) handles files in archiver temp folders that contain only one extracted file.

fn next() / fn prev()

Both delegate to step(direction). Return None when fewer than 2 images exist — navigation on a single-image directory is a silent no-op. Wrap around using wrapped_index().

fn peek_next() / fn peek_prev()

Read-only lookahead via peek(direction), used by preload_adjacent(). Returns None for single-image directories.

fn step(direction) / fn peek(direction) / fn wrapped_index(direction)

Three private helpers that eliminate duplication between next/prev and peek_next/peek_prev. wrapped_index computes the neighbour index with wraparound: forward uses modulo, backward uses checked_sub(1).unwrap_or(len - 1).

enum Direction

Forward and Backward. Passed to step, peek, and wrapped_index. More readable than a bool parameter at every call site.

fn remove(path)

Removes a path after deletion. Uses saturating_sub to keep the index in bounds when the deleted file was the last one.

fn rename(old_path, new_path)

Replaces a path in the sibling list after a successful rename. Preserves the current index so navigation order is unaffected.

fn rebuild()

Populates files from current_dir. Uses DirEntry::file_type() which is available from the readdir() result on every major OS at no extra syscall cost. is_file() would issue one additional stat() syscall per file — 5000 extra disk hits on a cold cache for a large folder.

fn human_sort / fn consume_number

Splits each filename into alternating text and digit runs. Digit runs are parsed as u64 and compared numerically, so img2.png sorts before img10.png. c.to_digit(10) is used in consume_number for clean digit extraction. Saturating arithmetic prevents overflow on pathological inputs.

4. loader.rs — Image Decoding

Owns the image type definitions and all decoding logic. Returns DecodedImage directly — no conversion needed in main.rs. No knowledge of winit, softbuffer, or UI state.

struct Frame

pixels: Vec<u8> — straight RGBA8, row-major. **delay_ms: u32** — frame duration in milliseconds. Zero for non-animated formats.

struct DecodedImage

One or more frames sharing the same canvas dimensions. **is_animated()** returns true when more than one frame exists. **frame(idx)** borrows a specific frame with a debug_assert on the index so out-of-range accesses panic with a useful message in debug builds. **single()** is a private constructor for the common single-frame case.

enum LoadOutcome

- **Ok(DecodedImage)** — decoded successfully using the declared extension.
- **WrongExtension(DecodedImage, String)** — decoded via header sniffing; the String is the correct extension (e.g. 'jpg'). The image is displayed immediately; the rename dialog is offered after the drain loop.
- **Unsupported(String)** — not a recognised image format even after header sniffing; the String is a human-readable reason.

fn load_image(path) → LoadOutcome

Two-pass strategy. Fast path: calls decode_by_ext with DecodeStrategy::ByExtension. This validates that the file's magic bytes agree with the extension before attempting a full decode. If the fast path fails, calls sniff_format. If a different format is detected, decodes with ByContent and returns WrongExtension. If the same extension is detected but decoding still fails, the file is corrupt. If no format is detected, returns Unsupported.

enum DecodeStrategy

- **ByExtension** — validates magic bytes against the declared extension, then calls image::open(). Fails on mismatch so WrongExtension is reached.
- **ByContent** — calls with_guessed_format() to decode purely from the file header. Used when the extension was already confirmed to be wrong.

■ *image::open() and with_guessed_format() both sniff the header internally and succeed silently on misnamed files. The ByExtension path must therefore perform an explicit magic-byte check before calling image::open().*

fn magic_to_format(buf) — single source of truth

Identifies an image format from its first 12 bytes. Used by both sniff_format (wrong-extension recovery) and decode_raster (extension validation), ensuring the two code paths are always consistent.

Format	Magic bytes	Length
JPEG	FF D8	2
PNG	89 50 4E 47	4
BMP	42 4D	2
GIF	47 49 46 38 (GIF8)	4
TIFF	49 49 2A 00 or 4D 4D 00 2A	4
ICO	00 00 01 00	4
WebP	52 49 46 46 ?? ?? ?? ?? 57 45 42 50 (RIFF....WEBP)	12

fn format_to_ext(fmt)

Maps an ImageFormat to the canonical extension string used throughout the module (e.g. ImageFormat::Jpeg → 'jpg'). Called by sniff_format to produce the extension string returned to load_image.

fn sniff_format(path)

Reads the first 32 bytes of the file once. Delegates to magic_to_format for raster formats via format_to_ext, then checks JXL signatures separately since JXL is not handled by the image crate's guesser.

fn read_magic(path)

Reads the first 12 bytes and returns the detected ImageFormat. Used by decode_raster on the ByExtension path to validate before decoding.

fn decode_raster — BMP, PNG, JPEG, WebP, TIFF, ICO

- **ByExtension**: derives declared format from the extension via ImageFormat::from_extension(), calls read_magic() for the actual format, returns Err immediately if they disagree. On match, calls image::open() which uses the extension for dispatch — the magic check already confirmed the content is correct.
- **ByContent**: opens a BufReader, calls with_guessed_format() which reads the header to select the decoder, then decode().

fn decode_gif — Animated GIF

Uses GifDecoder with AnimationDecoder. Corrupt individual frames are skipped (filter_map(ok)) rather than aborting the whole decode. Per-frame delay: 20 ms minimum to prevent zero-delay busy-loops; 100 ms fallback when the denominator is zero.

fn decode_jxl — JPEG XL

- Requests sRGB output regardless of embedded colour space.
- Guards against images larger than 32768×32768 before allocating.
- Uses checked_mul for the pixel_count calculation to catch overflow.
- Delegates conversion to jxl_samples_to_rgba8.

fn jxl_samples_to_rgba8

Converts interleaved float samples (1–4 channels) to RGBA8. Greyscale is broadcast to RGB; missing alpha defaults to fully opaque. Extracted from decode_jxl to give the conversion a clear name and testable boundary.

5. main.rs — Application Core

Contains the winit event loop, all application state, and the pixel renderer. No widget toolkit, no font renderer, no GPU pipeline.

5.1 Constants

- **BG: u8 = 42** — background grey for blank state, alpha compositing, and the pre-loop synchronous surface fill.
- **APP_NAME** — used in the title bar and all dialog captions.
- **DOUBLE_CLICK** — maximum gap between two left-clicks for double-click detection.

5.2 enum Msg — background thread messages

- **Loaded** — primary decode finished. Carries path, image, and OpenMode.
- **WrongExtension** — decoded via header sniff. Also carries the correct extension string so the UI can offer a rename dialog.
- **Preloaded** — speculative neighbour decode for the preload cache.
- **Unsupported** — file is not a recognised image format.
- **Deleted** — async deletion result: Ok(()) or Err(String).

5.3 struct DrainResult

- **resize_to** — non-None when the first image loaded; resize the window.
- **error_msg** — non-None when any error occurred (unsupported file or failed deletion). The window must be revealed before this dialog is shown.
- **rename_offer** — non-None when a file decoded under the wrong extension. The rename dialog is shown after the drain loop, never inside it.

5.4 struct App — fields

- **current_path / current_image** — the displayed file and its decoded pixels.
- **frame_index / next_frame_at** — GIF animation state.
- **navigator** — the DirNavigator for the current directory.
- **loading_path** — path currently being decoded. Used to discard stale Loaded messages when the user navigates quickly.
- **preload_cache** — decoded images for neighbouring files.
- **deleting** — prevents a second Delete keypress while deletion is in flight.
- **pending_center** — set when a resize is issued; cleared on the next WakeUp when the resize has landed and center_window() can be called accurately.
- **pre_maximize** — physical inner size saved before managed-maximise.
- **maximized / fullscreen** — independent boolean flags for the two window modes.
- **last_click** — timestamp for double-click detection via is_some_and().

5.5 fn toggle_maximize()

Saves window.inner_size() (physical pixels) before maximising. On restore, calls request_inner_size(size) and sets pending_center = true so center_window() fires asynchronously after the resize lands. No-op when fullscreen is true. Contains a debug_assert verifying that pre_maximize is Some whenever maximized is true — catching state desync in debug builds.

5.6 fn toggle_fullscreen() / fn exit_fullscreen()

toggle_fullscreen() sets Fullscreen::Borderless(None) or None based on the current state, then flips self.fullscreen. exit_fullscreen() returns false immediately if not in fullscreen so the event handler can skip the redraw.

5.7 fn open_image()

Cache hit: installs immediately via apply_image. Cache miss: seeds the navigator immediately (so peek_next/peek_prev are accurate for preloading), then spawns a background thread. The thread comment notes that send() and send_event() can only fail if the receiver is dropped, which means the UI thread has exited.

5.8 fn apply_image()

Stores the image, clears loading_path, updates the navigator, kicks off preload_adjacent(), and resets animation. Returns the image dimensions only when OpenMode::FirstImage so the event loop knows to resize the window.

5.9 fn preload_adjacent()

Spawns up to two background threads for the next and previous siblings. Only caches LoadOutcome::Ok — wrong-extension files would prompt a rename dialog and must not be silently preloaded. Does nothing for single-image directories.

5.10 fn tick_animation()

Uses let-else twice: first to short-circuit if next_frame_at is None, then to handle the (impossible in practice) case where current_image disappears between the animation tick and the frame advance.

5.11 fn on_deleted()

Returns Option<String> — the error message if deletion failed, None on success. Decouples the method from DrainResult so it has a single clear responsibility. The caller assigns the return value to dr.error_msg.

5.12 fn drain_messages()

Non-blocking try_recv loop. All blocking side-effects are returned in DrainResult. Contains a debug_assert checking that rename_offer is None before being set — catching the (impossible) case of two WrongExtension messages in one drain.

5.13 fn render()

Refactored for efficiency vs. the naive double loop:

- Rows entirely above or below the image are filled with row.fill(bg) — a single memset-like call rather than per-pixel branching.
- Left and right margins are filled with slice.fill(bg).
- inv_scale = 1.0 / scale is computed once per frame; multiplied per pixel instead of dividing — multiplication is faster on modern hardware.
- nearest() now takes width only (not height) and uses a slice bounds check via try_into() rather than an explicit clamp.
- composite() special-cases a == 255 (fully opaque) and a == 0 (fully transparent) to avoid the blend calculation for the common cases.
- blend_channel() is extracted so its purpose is named and it can be inlined independently.

5.14 fn offer_rename()

Probes writability by attempting to open the file with write access. This is the only reliable method on Windows: the parent directory's read-only attribute is set by convention on all shell folders (Desktop, Downloads, Pictures, etc.) to control Explorer's custom-view mechanism and does NOT indicate actual write protection. Checking the file itself avoids suppressing the rename dialog for every file in those folders.

5.15 Window helpers — refactored

- **work_area_rect()** — Windows-only helper returning the SPI_GETWORKAREA rectangle as (left, top, right, bottom) in physical pixels.
- **current_or_any_monitor()** — extracted helper to avoid repeating window.current_monitor().or_else(|| window.available_monitors().next()).
- **heuristic_work_area()** — 90% × 85% of the monitor, anchored at the monitor origin. Used as fallback on non-Windows and in center_window.
- **clamp_to_work_area()** — now measures the title bar height exactly from the live window frame (outer minus inner size) rather than a hardcoded value.
- **center_window()** — uses work_area_rect() directly on Windows for exact taskbar exclusion on any edge. Correctly clamps the bottom of the window to wa_bottom so the window is never partially below the taskbar.

5.16 fn image_size_hint()

Simplified: no longer has a special SVG branch (SVG support was removed). Opens the file and calls with_guessed_format() for all formats, which reads the magic bytes to determine dimensions without a full decode.

5.17 RedrawRequested — graceful surface error handling

Surface operations (resize, buffer_mut, present) can transiently fail during rapid window resizes. The event handler now uses is_err() returns and let-Ok patterns to skip the frame gracefully rather than panicking with .expect(). winit will issue another RedrawRequested once the window settles.

6. Threading & Asynchronous Design

6.1 The channel pipeline

- One bounded crossbeam channel, capacity 32.
- msg_tx is cloned cheaply for each background thread.
- msg_rx is drained non-blockingly on every WakeUp.
- Background threads post a Msg then call proxy.send_event(WakeUp).
- send() and send_event() failures are ignored — they only occur when the receiver is dropped, meaning the process is already exiting.

6.2 ControlFlow — zero idle CPU

ControlFlow::Wait sleeps until any event arrives. ControlFlow::WaitUntil wakes at the next GIF frame deadline. When displaying a still image the process uses effectively zero CPU.

6.3 Cold startup — header probe + async decode

image_size_hint() reads only the file header for dimensions so the window appears at the correct size with a gray background immediately. The full decode runs on a background thread. Startup is fast regardless of file size.

6.4 The two-WakeUp window reveal sequence

Window hidden → gray paint → decode thread starts → WakeUp arrives → resize issued → pending_center = true → self-WakeUp sent → next WakeUp: center_window() → set_visible(true) → RedrawRequested paints the image. The self-WakeUp is essential when opening from an archiver temp folder with only one file, because no preload threads fire to deliver the second WakeUp.

6.5 Stale result discarding

Loaded and WrongExtension messages are checked against loading_path. A mismatch means the user navigated away before the decode finished — the message is silently dropped.

6.6 Drain loop never blocks

All side-effects that could block (dialogs, rename) are collected in DrainResult and performed after the drain loop returns. on_deleted() returns Option<String> for the same reason.

6.7 Extension-mismatch recovery — how it works

decode_raster (ByExtension) reads 12 magic bytes via read_magic(), compares against the format implied by the extension, and returns Err immediately on mismatch. load_image() catches the Err, calls sniff_format(), and returns WrongExtension. The image is displayed immediately. The rename dialog appears after the drain loop.

■ *image::open() and with_guessed_format() both sniff the header and silently succeed on misnamed files. The explicit magic check in decode_raster is the only way to reliably detect extension mismatches.*

6.8 offer_rename() — writability probe

Probes writability by opening the file with write access before showing the dialog. On Windows, all shell folders (Desktop, Downloads, Pictures, etc.) have the read-only attribute set by convention — checking it would suppress the rename dialog for every file in those folders. Checking the file itself is reliable regardless of folder.

6.9 Rename collision avoidance

unique_rename_target() checks whether the target path already exists. If '5.jpg' already exists when renaming '5.png', the proposed name becomes '5 (1).jpg', matching Windows Explorer's convention.

6.10 Immediate process exit

Background threads hold msg_tx clones. A graceful exit would wait for all threads to finish. std::process::exit(0) terminates immediately, which is always correct for a stateless viewer with nothing to save.

6.11 The canonicalize trap

std::fs::canonicalize prepends \\?\ on Windows, which breaks the image crate and causes silent failures for archiver temp paths. The raw argument is used throughout.

7. Key Design Decisions & Trade-offs

winit + softbuffer over a GUI toolkit

A full toolkit adds ~200+ crates and ~9 MB. winit + softbuffer totals ~50–70 crates and ~2–3 MB. The trade-off: native OS dialogs must be called directly via windows-sys.

Magic-byte validation in decode_raster

The explicit 12-byte read and comparison in the ByExtension path is the only reliable way to detect extension mismatches. Every image-crate API that takes a path internally calls with_guessed_format(), which sniffs the header and silently succeeds regardless of the extension. The 12-byte read is negligible overhead compared to a full image decode.

magic_to_format as single source of truth

Both sniff_format (used for wrong-extension recovery) and decode_raster (used for extension validation) previously had separate detection logic. Consolidating into magic_to_format ensures they are always consistent and makes adding a new format a single-point change.

format_to_ext separation

Separating the ImageFormat → extension string mapping from magic_to_format keeps the magic table clean (it concerns bytes, not strings) and allows sniff_format to compose the two steps clearly.

DecodeStrategy enum over a bool parameter

decode_by_ext(&ext;, path, DecodeStrategy::ByContent) is self-documenting. decode_by_ext(&ext;, path, true) tells the reader nothing without looking up the function signature.

Direction enum in nav.rs

step(Direction::Forward) and step(Direction::Backward) read like prose. The three private helpers (step, peek, wrapped_index) eliminate the duplication between next/prev and peek_next/peek_prev.

current_dir field in DirNavigator

Previously the same-directory check inferred the current directory from files.first(). After the single-entry fallback replaced the list with one file, files.first() returned that single file's path — causing the next set_current to incorrectly see it as a different directory and rebuild. Storing current_dir explicitly makes the check authoritative.

DirEntry::file_type() over Path::is_file()

file_type() is available from the readdir() result on every major OS at no extra syscall cost. is_file() issues an additional stat() per file. On a cold cache with 5000 files that is 5000 extra disk hits.

render() row-based optimisation

Filling entire rows with bg in the margin areas uses a single slice::fill call per row rather than per-pixel branching. inv_scale precomputation and the composite() fast paths for a==0 and a==255 reduce per-pixel work for the common cases (fully opaque photographs, fully transparent borders).

clamp_to_work_area() title bar measurement

The title bar height is now measured from the live window frame (outer_size - inner_size) rather than a hardcoded constant, so it is correct regardless of DPI, Windows version, or custom themes.

offer_rename writability probe

Opening the file with write access is the only reliable portability-safe writability test. The parent directory read-only attribute is unsuitable on Windows because all shell folders carry it by convention.

8. Caveats & Known Limitations

No status bar

Filename, image index, and dimensions are not shown in the window. Showing text requires a font rasteriser. The filename is shown in the title bar.

Animated GIF only

Animation is supported for GIF. Animated WebP or AVIF would need separate decoder paths.

JXL HDR clamped

f32_to_u8 clamps values above 1.0 to 255, losing highlight detail for HDR JXL content. Standard JXL photos are unaffected.

Animated JXL

Only frame 0 is rendered. The file opens without error as a still image.

Work area heuristic on non-Windows

The 90% × 85% heuristic may not account for taskbars on non-Windows platforms accurately.

SC_MAXIMIZE winit behaviour dependency

The title-bar double-click is unified with the client-area double-click by relying on winit firing MouseInput for the title-bar gesture. This is observed behaviour rather than a documented guarantee.

Navigation from archiver temp folders

The temp folder typically contains only the one extracted file. Arrow keys, Page Up/Down, and mouse wheel are silent no-ops — correct behaviour.

9. Installer — devoid_viewer_setup.iss

Built with Inno Setup 6. Place the script in the project root, build the release binary first, then run iscc devoid_viewer_setup.iss. Output: Output\DevoidViewer_Setup.exe.

9.1 Why Inno Setup instead of WiX 3

WiX 3 evaluates Component Condition elements during the MSI costing phase, before any UI is shown. This makes per-user/per-machine scope switching impossible. Inno Setup handles it correctly via IsAdminInstallMode and HKA.

9.2 Installation scope

PrivilegesRequired=lowest with PrivilegesRequiredOverridesAllowed=dialog. Per-user by default (no UAC), with an optional per-machine choice (UAC). HKA resolves to HKLM or HKCU automatically.

9.3 App Paths registration

Registers DevoidViewer.exe in App Paths under HKLM or HKCU so Windows and archivers like 7-Zip can locate and launch it via ShellExecuteEx.

9.4 Supported extensions — 10 formats

jpg, jpeg, png, bmp, gif, webp, tiff, tif, ico, jxl. SVG was removed to keep the binary ultra-lightweight.

9.5 File associations

Feature	Registry written	Notes
ProgId	HKA\Software\Classes\DevoidViewer.ImageFile	Always installed.
Open With	HKA\...\OpenWithProgIds	Adds to Open With menu without changing the default.
Set as Default	HKLM\Software\Classes\.ext	Per-machine installs only (IsAdminInstallMode). Avoids U

9.6 The UserChoice problem

Writing the default association only to HKLM avoids generating UserChoice entries. UserChoice contains a cryptographic hash that Windows verifies on every file open. Stale entries pointing at a deleted ProgId survive uninstall and break file associations.

9.7 Uninstaller behaviour

- taskkill closes running instances before files are deleted.
- The install folder is always removed via uninsalwaysuninstall.
- All UserChoice keys for the 10 supported extensions are deleted on uninstall.
- Shell icon cache refreshed via ie4uinit.exe on install and uninstall.

10. Glossary of Rust Concepts Used

Ownership
Every value has exactly one owner. When the owner goes out of scope the value is dropped. No garbage collector needed.

Borrowing
Lend a reference (&T;) without transferring ownership. References cannot outlive the value they point to.

Option
Either Some(value) or None. The safe replacement for null pointers.

Result
Either Ok(value) or Err(error). The ? operator propagates errors up the call stack.

Vec
A growable heap-allocated array. Used for pixel buffers, frame lists, and path lists.

HashMap
A hash table. O(1) average lookup. Used for the preload cache keyed by PathBuf.

Arc
Atomically Reference Counted smart pointer. Shared ownership across threads. Used for the Window handle.

Enum with data
Rust enums carry data per variant. Msg::Loaded carries a PathBuf, a DecodedImage, and an OpenMode.

then_some
(condition).then_some(value) returns Some(value) if true, None otherwise. Used in apply_image for the resize decision.

is_some_and
option.is_some_and(|v| pred(v)) — combines is_some() and a predicate. Used in register_click for double-click detection.

let-else
let Ok(x) = expr else { return; } — early return on failure without nesting. Used throughout the event loop.

Lifetime
A compile-time label on a reference. View<'a> borrows from App — the compiler ensures View cannot outlive App.

Closure
An anonymous function. The move keyword transfers ownership of captured variables into the closure.

match
Exhaustive pattern matching. Every variant of an enum must be handled or the code will not compile.

debug_assert!
A panic in debug builds only. Used to catch state desync (maximized=true but pre_maximize=None) and impossible message sequences without any release overhead.

#[derive(...)]
Auto-generates trait implementations. #[derive(Clone, Debug)] writes clone() and the debug formatter.

Send
Marker trait meaning a type is safe to transfer to another thread.

ControlFlow
Winit enum controlling event loop sleep. Wait sleeps until any event. WaitUntil sleeps until a deadline.

EventLoopProxy
Thread-safe handle to the event loop. proxy.send_event() injects a synthetic event from any thread.

NonZeroU32
A u32 guaranteed non-zero at compile time. softbuffer's resize method requires this.

cfg(windows)
Conditional compilation. Code inside #[cfg(windows)] is compiled only on Windows.

unsafe
Opts into calling C API functions. Used for MessageBoxW, SystemParametersInfoW, and PostMessageW.

type alias
type BoxError = ...; creates a shorter name for Box with no runtime cost.

Zero-size struct
struct WakeUp; occupies zero bytes. Used as a type-level marker to wake the event loop.

include_bytes!
Embeds a file's raw bytes into the binary at compile time. Used for devoid.ico.

process::exit(0)
Terminates immediately, bypassing Rust's normal drop sequence. Correct for a stateless viewer.

DecodeStrategy
Enum replacing a bool parameter. ByExtension and ByContent are self-documenting at every call site.

Direction
Enum in nav.rs replacing a bool parameter for forward/backward navigation. Makes step() and peek() self-documenting.

magic_to_format
Single source of truth for magic-byte format detection. Used by both sniff_format and decode_raster to ensure consistency.